Skrypt szkoleniowy — Dzień 2

Modularny monolit w ASP.NET Core Web API (.NET 10)

Car Sharing • SQL Server 2022 • Autofac • MediatR • FluentValidation • EF Core

Temat przewodni: **SAGA, orkiestracja procesu i kompensacja**

Spis treści

1. Cel dnia drugiego
2. Punkt startowy
3. Wprowadzenie prowadzącego
4. Nowy scenariusz biznesowy
5. Najważniejszy przekaz dnia
6. Kiedy Saga ma sens
7. Choreografia i orkiestracja
8. Zmiana modelu rezerwacji
9. Kompensacja w module Fleet
10. Nowy moduł Payments
11. Kod modułu Payments
12. Modelowanie stanu Sagi
13. Encja stanu procesu
14. Rozszerzenie ReservationsDbContext
15. Komenda rozpoczynająca Sagę
16. Serwis orkiestrujący Sagę
17. Zdarzenia kończące proces
18. Handler startujący proces
19. Kontroler i endpoint procesu
20. Podgląd stanu Sagi
21. Reakcje modułu Notifications
22. Rejestracja Payments w hoście
23. Scenariusz sukcesu
24. Scenariusz porażki i kompensacji
25. Omówienie wzorca SAGA — tekst mówiony
26. Antywzorce
27. Powiązanie z mikroserwisami
28. Zadania warsztatowe
29. Podsumowanie dnia drugiego
30. Domknięcie szkolenia

---

1. Cel dnia drugiego

Celem drugiego dnia jest rozwinięcie aplikacji z dnia pierwszego tak, aby uczestnicy zobaczyli:

- jak przejść od prostego modularnego monolitu do świadomego modelowania procesu biznesowego,
- czym różni się pojedynczy use case od procesu wieloetapowego,
- dlaczego sama komunikacja zdarzeniowa nie wystarcza przy dłuższych przepływach,
- czym jest **Saga**,
- kiedy stosować **choreografię**, a kiedy **orkiestrację**,
- jak modelować **stan procesu**,
- jak reagować na błędy i robić **kompensację**,
- jak taki model przygotowuje modularny monolit do dalszej ewolucji.

> **Największą część dnia poświęcamy wzorcowi SAGA.**

---

2. Punkt startowy

Zakładamy, że po dniu pierwszym mamy działającą aplikację:

- moduł **Fleet**,
- moduł **Reservations**,
- moduł **Notifications**,
- komunikację in-process przez **MediatR**,
- rejestrację modułów przez **Autofac**,
- persystencję przez **EF Core + SQL Server**,
- publiczne API modułu Fleet.

Na dzień drugi dokładamy:

- moduł **Payments**,
- model procesu typu **Saga**,
- stan procesu,
- scenariusz kompensacji.

---

3. Wprowadzenie prowadzącego

Co mówi trener

> „Wczoraj zbudowaliśmy zdrowy fundament modularnego monolitu. Mieliśmy moduły, publiczne API, komunikację synchroniczną i zdarzeniową. To był bardzo dobry pierwszy krok.
>
> Dzisiaj chcę pokazać wam moment, w którym prosty use case zamienia się w proces biznesowy. I właśnie wtedy zaczynają się problemy, których nie rozwiąże sama dobra wola programisty ani pojedynczy handler.
>
> Jeżeli proces ma kilka kroków, obejmuje kilka modułów, ma punkty awarii i wymaga reakcji na niepowodzenie, to wchodzimy w obszar Sagi.”

---

4. Nowy scenariusz biznesowy

Wczorajszy proces

W dniu pierwszym rezerwacja wyglądała następująco:

1. sprawdzamy dostępność samochodu,
2. oznaczamy samochód jako niedostępny,
3. zapisujemy rezerwację,
4. publikujemy zdarzenie.

To był dobry przykład na początek, ale nadal dość prosty.

Dzisiejszy proces

Na dzień drugi rozszerzamy przepływ:

1. użytkownik składa żądanie rezerwacji,
2. system tworzy **rezerwację w statusie** `Pending`,
3. system rezerwuje samochód w module **Fleet**,
4. system próbuje autoryzować płatność w module **Payments**,
5. jeśli płatność się powiedzie:
    - rezerwacja przechodzi do statusu `Confirmed`,
    - publikujemy zdarzenie kończące proces,
6. jeśli płatność się nie powiedzie:
    - uwalniamy samochód,
    - rezerwacja przechodzi do statusu `Rejected`,
    - publikujemy zdarzenie o odrzuceniu rezerwacji,
    - zapisujemy ślad kompensacji.

Po co to robimy

Ten proces idealnie nadaje się do pokazania:

- stanu procesu,
- koordynacji kroków,
- punktów awarii,
- kompensacji,
- Sagi.

---

5. Najważniejszy przekaz dnia

Co mówi trener

> „Bardzo ważna rzecz: Saga nie jest magiczną transakcją rozproszoną. Saga nie sprawia, że wszystko stanie się atomowe. Saga pomaga zarządzać procesem, który jest rozbity na kilka kroków i kilka lokalnych decyzji.

> Jeżeli jeden krok się udał, a drugi nie, to Saga pomaga zdecydować, co dalej: czekamy, ponawiamy, kompensujemy, odrzucamy proces, przechodzimy do następnego stanu.
>
> To jest przede wszystkim wzorzec do zarządzania przepływem biznesowym, a nie biblioteka."

---

6. Kiedy Saga ma sens

Kiedy zwykle nie jest potrzebna

Saga zwykle **nie jest potrzebna**, gdy:

- mamy prosty CRUD,
- cały use case zamyka się w jednym module,
- jedna lokalna transakcja załatwia problem,
- nie ma prawdziwego procesu wieloetapowego.

Kiedy jest dobrym kandydatem

Saga jest dobrym kandydatem, gdy:

- proces trwa dłużej niż jeden prosty zapis,
- proces obejmuje kilka modułów,
- są punkty awarii pomiędzy krokami,
- nie możemy zrobić jednej wspólnej transakcji,
- potrzebujemy jawnego stanu procesu,
- potrzebujemy reakcji kompensacyjnych.

Co mówi trener

> „Jeżeli macie kilka kroków i między nimi mogą zdarzyć się błędy, to ktoś musi wiedzieć, na którym kroku jesteśmy, co już się udało i co trzeba zrobić, jeśli kolejny krok się nie powiedzie. I właśnie to robi Saga."

---

7. Choreografia i orkiestracja

7.1. Choreografia

W choreografii:

- moduły reagują na zdarzenia,
- nie ma jednego centralnego koordynatora,
- przepływ powstaje jako łańcuch reakcji.

Przykład

- Reservations publikuje `ReservationStarted`
- Fleet reaguje i publikuje `CarReservedForReservation`

- Payments reaguje i publikuje `PaymentAuthorized`
- Reservations reaguje i kończy proces

Zalety

- mało centralnej logiki,
- luźniejsze sprzężenie,
- dobre dla prostszych procesów.

Wady

- trudniej zrozumieć pełny proces,
- trudniej debugować,
- trudniej modelować timeouty i wyjątki,
- łatwo zgubić centralny obraz procesu.

7.2. Orkiestracja

W orkiestracji:

- istnieje centralny koordynator procesu,
- on zna aktualny stan,
- on decyduje o kolejnych krokach.

Zalety

- lepsza czytelność procesu,
- łatwiejsze modelowanie stanu,
- prostsze kompensacje,
- prostsze diagnozowanie błędów.

Wady

- więcej logiki w jednym miejscu,
- trzeba uważać, żeby koordynator nie stał się „bogiem procesu".

7.3. Decyzja szkoleniowa

Na szkoleniu pokazujemy **orkiestrację**, bo:

- jest czytelniejsza dydaktycznie,
- dobrze pokazuje stan procesu,
- łatwiej na niej omówić kompensację.

Co mówi trener

> „W realnych systemach możecie spotkać oba podejścia. Na szkoleniu wybieram orkiestrację, bo jest po prostu lepsza do nauki. Chcę, żebyście zobaczyli Sagę nie jako mglistą ideę, tylko jako bardzo konkretny model procesu."

---

8. Zmiana modelu rezerwacji

W dniu pierwszym `Reservation` od razu stawała się `Confirmed`.

Dzisiaj to zmieniamy:

- rezerwacja powstaje jako `Pending`,
- później przechodzi do `Confirmed` albo `Rejected`.

Kod encji `Reservation`

Plik: `CarSharing.Modules.Reservations/Domain/Reservation.cs`

```
namespace CarSharing.Modules.Reservations.Domain;

public sealed class Reservation
{
    public Guid Id { get; private set; }
    public Guid CarId { get; private set; }
    public Guid UserId { get; private set; }
    public DateTime FromUtc { get; private set; }
    public DateTime ToUtc { get; private set; }
    public string Status { get; private set; } = default!;

    private Reservation() { }

    public Reservation(Guid id, Guid carId, Guid userId, DateTime fromUtc,
DateTime toUtc)
    {
        Id = id;
        CarId = carId;
        UserId = userId;
        FromUtc = fromUtc;
        ToUtc = toUtc;
        Status = ReservationStatuses.Pending;
    }

    public void Confirm()
    {
        if (Status != ReservationStatuses.Pending)
        {
            throw new InvalidOperationException("Only pending reservation can be
confirmed.");
        }

        Status = ReservationStatuses.Confirmed;
    }

    public void Reject()
    {
        if (Status != ReservationStatuses.Pending)
```

```
        {
            throw new InvalidOperationException("Only pending reservation can be
rejected.");
        }

        Status = ReservationStatuses.Rejected;
    }
}

public static class ReservationStatuses
{
    public const string Pending = "Pending";
    public const string Confirmed = "Confirmed";
    public const string Rejected = "Rejected";
}
```

Co mówi trener

> „To jest bardzo ważna zmiana mentalna. Wczoraj mieliśmy use case. Dzisiaj mamy proces. A proces prawie zawsze oznacza, że w pewnym momencie obiekt biznesowy nie jest jeszcze zakończony. On jest w trakcie."

---

9. Kompensacja w module Fleet

Skoro płatność może się nie udać, moduł **Fleet** musi umieć:

- oznaczyć samochód jako niedostępny,
- oznaczyć samochód ponownie jako dostępny.

Publiczne API modułu Fleet

Plik: `CarSharing.Modules.Fleet/PublicApi/IFleetModuleApi.cs`

```
namespace CarSharing.Modules.Fleet.PublicApi;

public interface IFleetModuleApi
{
    Task<bool> IsCarAvailableAsync(Guid carId, CancellationToken cancellationToken
= default);
    Task MarkCarUnavailableAsync(Guid carId, CancellationToken cancellationToken =
default);
    Task MarkCarAvailableAsync(Guid carId, CancellationToken cancellationToken =
default);
}
```

Implementacja

Plik: `CarSharing.Modules.Fleet/PublicApi/FleetModuleApiService.cs`

```
using CarSharing.Modules.Fleet.Infrastructure;
using Microsoft.EntityFrameworkCore;

namespace CarSharing.Modules.Fleet.PublicApi;

public sealed class FleetModuleApiService(FleetDbContext dbContext) :
IFleetModuleApi
{
    public async Task<bool> IsCarAvailableAsync(Guid carId, CancellationToken
cancellationToken = default)
    {
        var car = await dbContext.Cars.SingleOrDefaultAsync(x => x.Id == carId,
cancellationToken);
        return car is not null && car.IsAvailable;
    }

    public async Task MarkCarUnavailableAsync(Guid carId, CancellationToken
cancellationToken = default)
    {
        var car = await dbContext.Cars.SingleOrDefaultAsync(x => x.Id == carId,
cancellationToken);

        if (car is null)
        {
            throw new InvalidOperationException("Car not found.");
        }

        car.MarkUnavailable();
        await dbContext.SaveChangesAsync(cancellationToken);
    }

    public async Task MarkCarAvailableAsync(Guid carId, CancellationToken
cancellationToken = default)
    {
        var car = await dbContext.Cars.SingleOrDefaultAsync(x => x.Id == carId,
cancellationToken);

        if (car is null)
        {
            throw new InvalidOperationException("Car not found.");
        }

        car.MarkAvailable();
        await dbContext.SaveChangesAsync(cancellationToken);
    }
}
```

Co mówi trener

> „To jest dobry moment, żeby pokazać sens kompensacji. Jeśli jeden krok procesu się nie powiedzie, nie cofamy czasu. Robimy nową, świadomą operację biznesową, która kompensuje wcześniejszy krok.”

---

10. Nowy moduł Payments

Rola modułu

Payments odpowiada za:

- próbę autoryzacji płatności,
- zapis próby płatności,
- zwrócenie wyniku: sukces albo porażka.

Nie interesuje go:

- pełna logika rezerwacji,
- flota,
- notyfikacje.

Co mówi trener

> „Payments jest dobrym modułem do pokazania punktu awarii. Możemy łatwo zasymulować sukces albo porażkę i dzięki temu zobaczyć, co robi Saga.”

Tworzenie projektu

```
dotnet new classlib -n CarSharing.Modules.Payments -o
src/Modules/CarSharing.Modules.Payments
dotnet sln add src/Modules/CarSharing.Modules.Payments

dotnet add src/CarSharing.Api reference src/Modules/CarSharing.Modules.Payments
dotnet add src/Modules/CarSharing.Modules.Payments reference
src/CarSharing.BuildingBlocks

# Reservations musi znać PublicApi Payments (ReservationSagaOrchestratorService
używa IPaymentsModuleApi)
dotnet add src/Modules/CarSharing.Modules.Reservations reference
src/Modules/CarSharing.Modules.Payments
```

Pakiety

```
dotnet add src/Modules/CarSharing.Modules.Payments package Autofac
dotnet add src/Modules/CarSharing.Modules.Payments package MediatR
dotnet add src/Modules/CarSharing.Modules.Payments package FluentValidation
dotnet add src/Modules/CarSharing.Modules.Payments package
Microsoft.EntityFrameworkCore
```

```
dotnet add src/Modules/CarSharing.Modules.Payments package
Microsoft.EntityFrameworkCore.SqlServer
```

---

11. Kod modułu Payments

Encja `PaymentAttempt`

Plik: `Domain/PaymentAttempt.cs`

```
namespace CarSharing.Modules.Payments.Domain;

public sealed class PaymentAttempt
{
    public Guid Id { get; private set; }
    public Guid ReservationId { get; private set; }
    public decimal Amount { get; private set; }
    public string Status { get; private set; } = default!;
    public DateTime CreatedAtUtc { get; private set; }

    private PaymentAttempt() { }

    public PaymentAttempt(Guid id, Guid reservationId, decimal amount, string
status, DateTime createdAtUtc)
    {
        Id = id;
        ReservationId = reservationId;
        Amount = amount;
        Status = status;
        CreatedAtUtc = createdAtUtc;
    }
}

public static class PaymentStatuses
{
    public const string Authorized = "Authorized";
    public const string Rejected = "Rejected";
}
```

`PaymentsDbContext`

Plik: `Infrastructure/PaymentsDbContext.cs`

```
using CarSharing.Modules.Payments.Domain;
using Microsoft.EntityFrameworkCore;

namespace CarSharing.Modules.Payments.Infrastructure;
```

```
public sealed class PaymentsDbContext(DbContextOptions<PaymentsDbContext> options)
: DbContext(options)
{
    public DbSet<PaymentAttempt> PaymentAttempts => Set<PaymentAttempt>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<PaymentAttempt>(entity =>
        {
            entity.ToTable("payment_attempts");
            entity.HasKey(x => x.Id);

            entity.Property(x => x.ReservationId).IsRequired();
            entity.Property(x => x.Amount).HasColumnType("decimal(18,2)");
            entity.Property(x => x.Status).HasMaxLength(30).IsRequired();
            entity.Property(x => x.CreatedAtUtc).IsRequired();
        });
    }
}
```

Publiczne API modułu Payments

Plik: `PublicApi/IPaymentsModuleApi.cs`

```
namespace CarSharing.Modules.Payments.PublicApi;

public interface IPaymentsModuleApi
{
    Task<PaymentAuthorizationResult> AuthorizeAsync(
        Guid reservationId,
        decimal amount,
        CancellationToken cancellationToken = default);
}

public sealed record PaymentAuthorizationResult(
    bool IsAuthorized,
    string Reason);
```

Implementacja API

Plik: `PublicApi/PaymentsModuleApiService.cs`

```
using CarSharing.Modules.Payments.Domain;
using CarSharing.Modules.Payments.Infrastructure;

namespace CarSharing.Modules.Payments.PublicApi;

public sealed class PaymentsModuleApiService(PaymentsDbContext dbContext) :
IPaymentsModuleApi
```

```
{
    public async Task<PaymentAuthorizationResult> AuthorizeAsync(
        Guid reservationId,
        decimal amount,
        CancellationToken cancellationToken = default)
    {
        var isAuthorized = amount <= 300m;

        var attempt = new PaymentAttempt(
            Guid.NewGuid(),
            reservationId,
            amount,
            isAuthorized ? PaymentStatuses.Authorized : PaymentStatuses.Rejected,
            DateTime.UtcNow);

        dbContext.PaymentAttempts.Add(attempt);
        await dbContext.SaveChangesAsync(cancellationToken);

        return isAuthorized
            ? new PaymentAuthorizationResult(true, "Payment authorized.")
            : new PaymentAuthorizationResult(false, "Payment rejected by business
rule.");
    }
}
```

PaymentsModule

Plik: PaymentsModule.cs

```
using Autofac;
using CarSharing.Modules.Payments.Infrastructure;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;

namespace CarSharing.Modules.Payments;

public sealed class PaymentsModule(IConfiguration configuration) : Module
{
    protected override void Load(ContainerBuilder builder)
    {
        builder.Register(ctx =>
            {
                var options = new DbContextOptionsBuilder<PaymentsDbContext>()

.UseSqlServer(configuration.GetConnectionString("PaymentsDb")!)
                    .Options;

                return new PaymentsDbContext(options);
            })
            .AsSelf()
            .InstancePerLifetimeScope();
```

```
        builder.RegisterAssemblyTypes(ThisAssembly)
            .Where(t =>
                t.Name.EndsWith("Service", StringComparison.Ordinal) &&
                t.Namespace is not null &&
                t.Namespace.StartsWith("CarSharing.Modules.Payments",
StringComparison.Ordinal))
            .AsImplementedInterfaces()
            .InstancePerLifetimeScope();
    }
}
```

Co mówi trener

> „Tutaj symulujemy decyzję płatniczą bardzo prosto. Chodzi nam nie o integrację z realnym providerem płatności, tylko o kontrolowany punkt sukcesu i porażki, który uruchomi różne ścieżki Sagi.”

---

12. Modelowanie stanu Sagi

Dla czytelności wprowadzamy nowy element: **ReservationSaga**.

To nie musi być osobny proces ani osobny serwis. W modularnym monolicie może to być:

- encja stanu procesu,
- serwis orkiestracyjny,
- handler komendy rozpoczynającej proces.

Co Saga musi wiedzieć

Saga musi wiedzieć:

- jaki proces prowadzi,
- w jakim jest stanie,
- co już się udało,
- co trzeba zrobić dalej,
- co zrobić przy błędzie.

Stany Sagi

Proponowane stany:

- `Started`
- `CarReserved`
- `PaymentAuthorized`
- `Completed`
- `Compensating`
- `Rejected`

Co mówi trener

> „Największy błąd przy nauce Sagi to myślenie o niej jako o sekwencji ifów. Saga to nie tylko ‘zrób krok A, potem krok B’. Saga to również jawny stan procesu. I bez tego bardzo szybko gubimy kontrolę nad tym, co już się wydarzyło.”

---

13. Encja stanu procesu

ReservationSagaState

Plik: `CarSharing.Modules.Reservations/Domain/ReservationSagaState.cs`

```
namespace CarSharing.Modules.Reservations.Domain;

public sealed class ReservationSagaState
{
    public Guid Id { get; private set; }
    public Guid ReservationId { get; private set; }
    public Guid CarId { get; private set; }
    public Guid UserId { get; private set; }
    public decimal Price { get; private set; }
    public string State { get; private set; } = default!;
    public DateTime CreatedAtUtc { get; private set; }
    public DateTime? UpdatedAtUtc { get; private set; }

    private ReservationSagaState() { }

    public ReservationSagaState(
        Guid id,
        Guid reservationId,
        Guid carId,
        Guid userId,
        decimal price,
        DateTime createdAtUtc)
    {
        Id = id;
        ReservationId = reservationId;
        CarId = carId;
        UserId = userId;
        Price = price;
        CreatedAtUtc = createdAtUtc;
        State = ReservationSagaStates.Started;
    }

    public void MarkCarReserved()
    {
        State = ReservationSagaStates.CarReserved;
        UpdatedAtUtc = DateTime.UtcNow;
    }
```

```
    public void MarkPaymentAuthorized()
    {
        State = ReservationSagaStates.PaymentAuthorized;
        UpdatedAtUtc = DateTime.UtcNow;
    }

    public void MarkCompleted()
    {
        State = ReservationSagaStates.Completed;
        UpdatedAtUtc = DateTime.UtcNow;
    }

    public void MarkCompensating()
    {
        State = ReservationSagaStates.Compensating;
        UpdatedAtUtc = DateTime.UtcNow;
    }

    public void MarkRejected()
    {
        State = ReservationSagaStates.Rejected;
        UpdatedAtUtc = DateTime.UtcNow;
    }
}

public static class ReservationSagaStates
{
    public const string Started = "Started";
    public const string CarReserved = "CarReserved";
    public const string PaymentAuthorized = "PaymentAuthorized";
    public const string Completed = "Completed";
    public const string Compensating = "Compensating";
    public const string Rejected = "Rejected";
}
```

Co mówi trener

> „Zwróćcie uwagę, że stan Sagi to nie to samo co stan samej rezerwacji. Rezerwacja jest obiektem biznesowym dla klienta. Saga jest obiektem techniczno-biznesowym, który śledzi proces.”

---

14. Rozszerzenie `ReservationsDbContext`

Dodanie tabeli Sagi

Plik: `CarSharing.Modules.Reservations/Infrastructure/ReservationsDbContext.cs`

```
using CarSharing.Modules.Reservations.Domain;
using Microsoft.EntityFrameworkCore;
```

```
namespace CarSharing.Modules.Reservations.Infrastructure;

public sealed class ReservationsDbContext(DbContextOptions<ReservationsDbContext>
options) : DbContext(options)
{
    public DbSet<Reservation> Reservations => Set<Reservation>();
    public DbSet<ReservationSagaState> ReservationSagaStates =>
Set<ReservationSagaState>();

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        modelBuilder.Entity<Reservation>(entity =>
        {
            entity.ToTable("reservations");
            entity.HasKey(x => x.Id);

            entity.Property(x => x.CarId).IsRequired();
            entity.Property(x => x.UserId).IsRequired();
            entity.Property(x => x.FromUtc).IsRequired();
            entity.Property(x => x.ToUtc).IsRequired();
            entity.Property(x => x.Status).HasMaxLength(30).IsRequired();
        });

        modelBuilder.Entity<ReservationSagaState>(entity =>
        {
            entity.ToTable("reservation_saga_states");
            entity.HasKey(x => x.Id);

            entity.Property(x => x.ReservationId).IsRequired();
            entity.Property(x => x.CarId).IsRequired();
            entity.Property(x => x.UserId).IsRequired();
            entity.Property(x => x.Price).HasColumnType("decimal(18,2)");
            entity.Property(x => x.State).HasMaxLength(50).IsRequired();
            entity.Property(x => x.CreatedAtUtc).IsRequired();
            entity.Property(x => x.UpdatedAtUtc).IsRequired(false);

            entity.HasIndex(x => x.ReservationId).IsUnique();
        });
    }
}
```

---

15. Komenda rozpoczynająca Sagę

StartReservationSagaCommand

Plik: `Application/StartReservationSaga/StartReservationSagaCommand.cs`

```
using MediatR;
```

```
namespace CarSharing.Modules.Reservations.Application.StartReservationSaga;

public sealed record StartReservationSagaCommand(
    Guid CarId,
    Guid UserId,
    DateTime FromUtc,
    DateTime ToUtc,
    decimal Price) : IRequest<Guid>;
```

Walidator

Plik: `Application/StartReservationSaga/StartReservationSagaCommandValidator.cs`

```
using FluentValidation;

namespace CarSharing.Modules.Reservations.Application.StartReservationSaga;

public sealed class StartReservationSagaCommandValidator :
AbstractValidator<StartReservationSagaCommand>
{
    public StartReservationSagaCommandValidator()
    {
        RuleFor(x => x.CarId).NotEmpty();
        RuleFor(x => x.UserId).NotEmpty();
        RuleFor(x => x.Price).GreaterThan(0);
        RuleFor(x => x.FromUtc).LessThan(x => x.ToUtc);
    }
}
```

---

16. Serwis orkiestrujący Sagę

Interfejs

Plik: `Application/ReservationSaga/IReservationSagaOrchestratorService.cs`

```
namespace CarSharing.Modules.Reservations.Application.ReservationSaga;

public interface IReservationSagaOrchestratorService
{
    Task<Guid> StartAsync(
        Guid carId,
        Guid userId,
        DateTime fromUtc,
        DateTime toUtc,
        decimal price,
```

```
        CancellationToken cancellationToken = default);
}
```

Implementacja

Plik: `Application/ReservationSaga/ReservationSagaOrchestratorService.cs`

```
using CarSharing.Modules.Fleet.PublicApi;
using CarSharing.Modules.Payments.PublicApi;
using CarSharing.Modules.Reservations.Contracts;
using CarSharing.Modules.Reservations.Domain;
using CarSharing.Modules.Reservations.Infrastructure;
using MediatR;

namespace CarSharing.Modules.Reservations.Application.ReservationSaga;

public sealed class ReservationSagaOrchestratorService(
    ReservationsDbContext dbContext,
    IFleetModuleApi fleetModuleApi,
    IPaymentsModuleApi paymentsModuleApi,
    IPublisher publisher)
    : IReservationSagaOrchestratorService
{
    public async Task<Guid> StartAsync(
        Guid carId,
        Guid userId,
        DateTime fromUtc,
        DateTime toUtc,
        decimal price,
        CancellationToken cancellationToken = default)
    {
        var isAvailable = await fleetModuleApi.IsCarAvailableAsync(carId,
cancellationToken);

        if (!isAvailable)
        {
            throw new InvalidOperationException("Car is not available.");
        }

        var reservation = new Reservation(Guid.NewGuid(), carId, userId, fromUtc,
toUtc);
        dbContext.Reservations.Add(reservation);

        var saga = new ReservationSagaState(
            Guid.NewGuid(),
            reservation.Id,
            reservation.CarId,
            reservation.UserId,
            price,
            DateTime.UtcNow);
```

```
        dbContext.ReservationSagaStates.Add(saga);
        await dbContext.SaveChangesAsync(cancellationToken);

        await fleetModuleApi.MarkCarUnavailableAsync(carId, cancellationToken);
        saga.MarkCarReserved();
        await dbContext.SaveChangesAsync(cancellationToken);

        var paymentResult = await paymentsModuleApi.AuthorizeAsync(reservation.Id,
price, cancellationToken);

        if (paymentResult.IsAuthorized)
        {
            saga.MarkPaymentAuthorized();
            reservation.Confirm();

            saga.MarkCompleted();
            await dbContext.SaveChangesAsync(cancellationToken);

            await publisher.Publish(
                new ReservationConfirmed(
                    reservation.Id,
                    reservation.CarId,
                    reservation.UserId,
                    reservation.FromUtc,
                    reservation.ToUtc,
                    price,
                    DateTime.UtcNow),
                cancellationToken);

            return reservation.Id;
        }

        saga.MarkCompensating();
        await dbContext.SaveChangesAsync(cancellationToken);

        await fleetModuleApi.MarkCarAvailableAsync(carId, cancellationToken);

        reservation.Reject();
        saga.MarkRejected();

        await dbContext.SaveChangesAsync(cancellationToken);

        await publisher.Publish(
            new ReservationRejected(
                reservation.Id,
                reservation.CarId,
                reservation.UserId,
                price,
                paymentResult.Reason,
                DateTime.UtcNow),
            cancellationToken);

        return reservation.Id;
```

```
    }
}
```

Co mówi trener

> „To jest serce dzisiejszego dnia. Tutaj dokładnie widać, czym Saga różni się od zwykłego handlera use case'a.
>
> Najpierw tworzymy obiekt biznesowy i stan procesu. Potem wykonujemy krok po kroku kolejne działania. Po każdym ważnym etapie aktualizujemy stan Sagi. A jeśli płatność się nie powiedzie, wchodzimy w kompensację.
>
> To jest moment, w którym bardzo dobrze widać różnicę między 'wszystko się udało' a 'część się udała, część nie'. Saga daje nam model na taki świat."

---

17. Zdarzenia kończące proces

`ReservationConfirmed`

Plik: `Contracts/ReservationConfirmed.cs`

```
using MediatR;

namespace CarSharing.Modules.Reservations.Contracts;

public sealed record ReservationConfirmed(
    Guid ReservationId,
    Guid CarId,
    Guid UserId,
    DateTime FromUtc,
    DateTime ToUtc,
    decimal Price,
    DateTime OccurredOnUtc) : INotification;
```

`ReservationRejected`

Plik: `Contracts/ReservationRejected.cs`

```
using MediatR;

namespace CarSharing.Modules.Reservations.Contracts;

public sealed record ReservationRejected(
    Guid ReservationId,
    Guid CarId,
    Guid UserId,
```

```
    decimal Price,
    string Reason,
    DateTime OccurredOnUtc) : INotification;
```

Co mówi trener

> „Wczoraj mieliśmy jedno zdarzenie. Dzisiaj rozbijamy zakończenie procesu na dwa świadome fakty biznesowe: potwierdzenie albo odrzucenie rezerwacji. To bardzo ważne, bo inne moduły mogą reagować inaczej na każdy z tych wariantów.”

---

18. Handler startujący proces

`StartReservationSagaCommandHandler`

Plik: `Application/StartReservationSaga/StartReservationSagaCommandHandler.cs`

```
using CarSharing.Modules.Reservations.Application.ReservationSaga;
using MediatR;

namespace CarSharing.Modules.Reservations.Application.StartReservationSaga;

public sealed class StartReservationSagaCommandHandler(
    IReservationSagaOrchestratorService orchestratorService)
    : IRequestHandler<StartReservationSagaCommand, Guid>
{
    public Task<Guid> Handle(StartReservationSagaCommand request,
CancellationToken cancellationToken)
    {
        return orchestratorService.StartAsync(
            request.CarId,
            request.UserId,
            request.FromUtc,
            request.ToUtc,
            request.Price,
            cancellationToken);
    }
}
```

---

19. Kontroler i endpoint procesu

Rozszerzenie `ReservationsController`

```
using CarSharing.Modules.Reservations.Application.GetReservationSagaStates;
using CarSharing.Modules.Reservations.Application.GetReservations;
```

```
using CarSharing.Modules.Reservations.Application.StartReservationSaga;
using MediatR;
using Microsoft.AspNetCore.Mvc;

namespace CarSharing.Modules.Reservations.Controllers;

[ApiController]
[Route("api/reservations")]
public sealed class ReservationsController(ISender sender) : ControllerBase
{
    [HttpPost("process")]
    public async Task<IActionResult> StartProcess(
        [FromBody] StartReservationSagaCommand command,
        CancellationToken cancellationToken)
    {
        var reservationId = await sender.Send(command, cancellationToken);
        return Ok(new { ReservationId = reservationId });
    }

    [HttpGet]
    public async Task<IActionResult> GetReservations(CancellationToken
cancellationToken)
    {
        var result = await sender.Send(new GetReservationsQuery(),
cancellationToken);
        return Ok(result);
    }

    [HttpGet("processes")]
    public async Task<IActionResult> GetProcesses(CancellationToken
cancellationToken)
    {
        var result = await sender.Send(new GetReservationSagaStatesQuery(),
cancellationToken);
        return Ok(result);
    }
}
```

Co mówi trener

> „Zobaczcie, że endpoint nazywa się inaczej. Celowo pokazuję, że to nie jest już zwykłe ‘utwórz rezerwację’. To jest uruchomienie procesu. Ta zmiana w nazewnictwie pomaga zespołowi myśleć architektonicznie.”

---

20. Podgląd stanu Sagi

`GetReservationSagaStatesQuery`

Plik: `Application/GetReservationSagaStates/GetReservationSagaStatesQuery.cs`

```
using MediatR;

namespace CarSharing.Modules.Reservations.Application.GetReservationSagaStates;

public sealed record GetReservationSagaStatesQuery()
    : IRequest<IReadOnlyList<ReservationSagaStateDto>>;

public sealed record ReservationSagaStateDto(
    Guid ReservationId,
    string State,
    decimal Price,
    DateTime CreatedAtUtc,
    DateTime? UpdatedAtUtc);
```

Handler

Plik: `Application/GetReservationSagaStates/GetReservationSagaStatesQueryHandler.cs`

```
using CarSharing.Modules.Reservations.Infrastructure;
using MediatR;
using Microsoft.EntityFrameworkCore;

namespace CarSharing.Modules.Reservations.Application.GetReservationSagaStates;

public sealed class GetReservationSagaStatesQueryHandler(ReservationsDbContext
dbContext)
    : IRequestHandler<GetReservationSagaStatesQuery,
IReadOnlyList<ReservationSagaStateDto>>
{
    public async Task<IReadOnlyList<ReservationSagaStateDto>> Handle(
        GetReservationSagaStatesQuery request,
        CancellationToken cancellationToken)
    {
        return await dbContext.ReservationSagaStates
            .AsNoTracking()
            .OrderByDescending(x => x.CreatedAtUtc)
            .Select(x => new ReservationSagaStateDto(
                x.ReservationId,
                x.State,
                x.Price,
                x.CreatedAtUtc,
                x.UpdatedAtUtc))
            .ToListAsync(cancellationToken);
    }
}
```

Co mówi trener

> „To jest bardzo ważny element dydaktyczny. Bez możliwości obejrzenia stanu procesu Saga szybko robi się pojęciem abstrakcyjnym. Ja chcę, żebyście mogli zobaczyć ten stan wprost."

---

21. Reakcje modułu Notifications

Handler sukcesu

Plik: `Application/ReservationConfirmedNotificationHandler.cs`

```
using CarSharing.Modules.Notifications.Domain;
using CarSharing.Modules.Notifications.Infrastructure;
using CarSharing.Modules.Reservations.Contracts;
using MediatR;

namespace CarSharing.Modules.Notifications.Application;

public sealed class ReservationConfirmedNotificationHandler(NotificationsDbContext
dbContext)
    : INotificationHandler<ReservationConfirmed>
{
    public async Task Handle(ReservationConfirmed notification, CancellationToken
cancellationToken)
    {
        var message =
            $"Reservation {notification.ReservationId} confirmed for user
{notification.UserId}. Price: {notification.Price}.";

        dbContext.NotificationLogs.Add(
            new NotificationLog(Guid.NewGuid(), message, DateTime.UtcNow));

        await dbContext.SaveChangesAsync(cancellationToken);
    }
}
```

Handler porażki

Plik: `Application/ReservationRejectedNotificationHandler.cs`

```
using CarSharing.Modules.Notifications.Domain;
using CarSharing.Modules.Notifications.Infrastructure;
using CarSharing.Modules.Reservations.Contracts;
using MediatR;

namespace CarSharing.Modules.Notifications.Application;

public sealed class ReservationRejectedNotificationHandler(NotificationsDbContext
dbContext)
    : INotificationHandler<ReservationRejected>
```

```
{
    public async Task Handle(ReservationRejected notification, CancellationToken
cancellationToken)
    {
        var message =
            $"Reservation {notification.ReservationId} rejected. Reason:
{notification.Reason}.";

        dbContext.NotificationLogs.Add(
            new NotificationLog(Guid.NewGuid(), message, DateTime.UtcNow));

        await dbContext.SaveChangesAsync(cancellationToken);
    }
}
```

Co mówi trener

> „Teraz naprawdę widać przewagę tego podejścia. Moduł Notifications nie zna orkiestratora Sagi, nie zna Payments, nie zna szczegółów Fleet. On po prostu reaguje na świadomie nazwane fakty biznesowe.”

---

22. Rejestracja Payments w hoście

Rozszerzenie `Program.cs`

```
using CarSharing.Modules.Payments;
using CarSharing.Modules.Payments.Infrastructure;

builder.Services.AddMediatR(cfg =>
{
    cfg.RegisterServicesFromAssemblies(
        typeof(Program).Assembly,
        typeof(FleetModule).Assembly,
        typeof(ReservationsModule).Assembly,
        typeof(NotificationsModule).Assembly,
        typeof(PaymentsModule).Assembly);
});

builder.Host.ConfigureContainer<ContainerBuilder>(containerBuilder =>
{
    containerBuilder.RegisterModule(new FleetModule(builder.Configuration));
    containerBuilder.RegisterModule(new
ReservationsModule(builder.Configuration));
    containerBuilder.RegisterModule(new
NotificationsModule(builder.Configuration));
    containerBuilder.RegisterModule(new PaymentsModule(builder.Configuration));
});
```

```
using (var scope = app.Services.CreateScope())
{
    scope.ServiceProvider.GetRequiredService<FleetDbContext>
().Database.EnsureCreated();
    scope.ServiceProvider.GetRequiredService<ReservationsDbContext>
().Database.EnsureCreated();
    scope.ServiceProvider.GetRequiredService<NotificationsDbContext>
().Database.EnsureCreated();
    scope.ServiceProvider.GetRequiredService<PaymentsDbContext>
().Database.EnsureCreated();
}
```

appsettings.json

```
{
  "ConnectionStrings": {
    "FleetDb":
"Server=localhost;Database=CarSharingFleet;Trusted_Connection=True;TrustServerCert
ificate=True;",
    "ReservationsDb":
"Server=localhost;Database=CarSharingReservations;Trusted_Connection=True;TrustSer
verCertificate=True;",
    "NotificationsDb":
"Server=localhost;Database=CarSharingNotifications;Trusted_Connection=True;TrustSe
rverCertificate=True;",
    "PaymentsDb":
"Server=localhost;Database=CarSharingPayments;Trusted_Connection=True;TrustServerC
ertificate=True;"
  }
}
```

---

23. Scenariusz sukcesu

Krok 1 — dodanie auta

```
POST /api/fleet/cars
Content-Type: application/json

{
  "brand": "Toyota",
  "model": "Corolla",
  "registrationNumber": "WX20001"
}
```

Krok 2 — uruchomienie procesu z ceną 120

```
POST /api/reservations/process
Content-Type: application/json

{
  "carId": "TU_WKLEJ_ID_AUTA",
  "userId": "11111111-1111-1111-1111-111111111111",
  "fromUtc": "2026-04-21T08:00:00Z",
  "toUtc": "2026-04-21T10:00:00Z",
  "price": 120.00
}
```

Co sprawdzamy

- `GET /api/reservations`
- `GET /api/reservations/processes`
- `GET /api/notifications/logs`

Oczekiwany wynik

- rezerwacja w statusie `Confirmed`,
- Saga w stanie `Completed`,
- log potwierdzenia.

Co mówi trener

> „To jest ścieżka szczęśliwa. Wszystko się udało. Ale dla Sagi najciekawsze nie jest to, że wszystko zadziałało. Najciekawsze jest to, co się dzieje, gdy proces się łamie w połowie.”

---

24. Scenariusz porażki i kompensacji

Krok 1 — dodanie drugiego auta

```
POST /api/fleet/cars
Content-Type: application/json

{
  "brand": "Kia",
  "model": "Ceed",
  "registrationNumber": "WX20002"
}
```

Krok 2 — uruchomienie procesu z ceną 500

```
POST /api/reservations/process
Content-Type: application/json

{
  "carId": "TU_WKLEJ_ID_AUTA",
  "userId": "22222222-2222-2222-2222-222222222222",
  "fromUtc": "2026-04-21T12:00:00Z",
  "toUtc": "2026-04-21T16:00:00Z",
  "price": 500.00
}
```

Oczekiwany wynik

- płatność odrzucona,
- samochód ponownie dostępny,
- rezerwacja w statusie `Rejected`,
- Saga w stanie `Rejected`,
- log odrzucenia,
- kompensacja wykonana.

Co mówi trener

> „Tutaj właśnie widać, po co nam Saga. Część procesu już się wydarzyła. Auto zostało zarezerwowane. Potem płatność się nie udała. I teraz potrzebujemy logicznie domknąć proces. Nie przez cofnięcie czasu, tylko przez świadomą kompensację.”

---

25. Omówienie wzorca SAGA — tekst mówiony

Czym Saga jest

> „Saga jest wzorcem do zarządzania procesem wieloetapowym. Nie daje jednej wielkiej transakcji. Zamiast tego dzieli proces na serię kroków i lokalnych decyzji.”

Czym Saga nie jest

> „Saga nie jest magicznym rollbackiem wszystkiego. Jeśli płatność się nie udała, a auto zostało już oznaczone jako niedostępne, to nie cofamy historii. Robimy kompensację: oznaczamy auto z powrotem jako dostępne.”

Dlaczego stan jest kluczowy

> „Bez jawnego stanu Sagi bardzo szybko przestajemy wiedzieć, co już się udało, co czeka na wykonanie i co trzeba naprawić. Stan procesu jest jednym z najważniejszych elementów całego wzorca.”

Dlaczego orkiestracja jest dobra na początek

> „Orkiestracja daje nam czytelność. W jednym miejscu widać logikę procesu. Na początek to jest ogromna zaleta. Choreografia bywa elegancka, ale dużo łatwiej się w niej zgubić."

Co jest najtrudniejsze w praktyce

> „Najtrudniejsze w praktyce nie jest samo napisanie ifów. Najtrudniejsze jest dobre nazwanie stanów, kroków, zdarzeń i kompensacji. Czyli tak naprawdę najtrudniejsze jest modelowanie procesu biznesowego."

---

26. Antywzorce

Antywzorzec 1 — Saga bez stanu

Nie robimy czegoś takiego:

```
if (step1) { }
if (step2) { }
if (step3) { }
```

bez żadnego jawnego modelu procesu.

Co mówi trener

> „Jeżeli nie zapisujecie stanu procesu, to nie macie prawdziwej Sagi. Macie dłuższy handler."

Antywzorzec 2 — zbyt wiele logiki ukrytej w zdarzeniach

Co mówi trener

> „Jeżeli cały proces jest rozsmarowany po sześciu handlerach zdarzeń i nikt nie umie odpowiedzieć, co dokładnie dzieje się w scenariuszu błędu, to znak, że choreografia jest zbyt rozproszona."

Antywzorzec 3 — kompensacja jako usuwanie danych

Co mówi trener

> „Kompensacja nie polega na usuwaniu śladów po tym, że coś się wydarzyło. Kompensacja polega na wykonaniu nowej, sensownej operacji, która biznesowo niweluje wcześniejszy efekt."

Antywzorzec 4 — jeden gigantyczny orchestrator

Co mówi trener

> „Trzeba też uważać, żeby orkiestrator nie stał się wielkim centrum całej domeny. Jeśli jeden orchestrator steruje wszystkim w systemie, to po prostu tworzymy nowy monolit w monolicie."

---

27. Powiązanie z mikroserwisami

Co mówi trener

> „To, co budujemy dzisiaj, nadal działa w jednym procesie. Ale sposób myślenia jest już bardzo podobny do tego, co później widzimy w architekturach bardziej rozproszonych.
>
> Mamy moduły. Mamy publiczne API. Mamy zdarzenia. Mamy proces wieloetapowy. Mamy kompensację. To wszystko są elementy, które później można przenieść dalej, jeśli kiedyś zapadnie decyzja o wydzielaniu części systemu.
>
> Właśnie dlatego modularny monolit jest tak dobrym etapem dojrzewania architektury.”

---

28. Zadania warsztatowe

Zadanie 1

Dodaj timeout procesu:

- jeśli proces pozostaje w stanie `Started` zbyt długo, oznacz go jako `Rejected`.

Zadanie 2

Dodaj nowy stan:

- `PaymentRejected`

i pokaż różnicę między:

- błędem płatności,
- kompensacją,
- stanem finalnym.

Zadanie 3

Dodaj moduł `Pricing`, który:

- obliczy cenę przed autoryzacją płatności,
- a Saga wykorzysta jego wynik.

Zadanie 4

Zmień orkiestrację na uproszczoną choreografię i porównaj:

- czytelność,
- łatwość debugowania,
- miejsce podejmowania decyzji.

Co mówi trener

> „Te ćwiczenia są bardzo ważne, bo pomagają zobaczyć, że Saga to nie jednorazowy kod, tylko sposób myślenia o procesie.”

---

29. Podsumowanie dnia drugiego

Co mówi trener

> „Dzisiaj przeszliśmy od prostego use case’a do prawdziwego procesu biznesowego. I to jest duża zmiana jakościowa.
>
> Pokazaliśmy, że samo publikowanie zdarzeń nie wystarcza, kiedy proces ma kilka kroków, punkty awarii i potrzebę kompensacji. Wprowadziliśmy stan procesu, orkiestrację, kompensację i świadome zdarzenia kończące proces.
>
> Największy wniosek, jaki chciałbym wam zostawić, jest taki: Saga nie jest sztuczką techniczną. Saga jest sposobem modelowania złożonego procesu biznesowego.
>
> A kiedy proces jest dobrze zamodelowany, technologia zaczyna być wsparciem, a nie chaosem.”

---

30. Domknięcie szkolenia

Na koniec można płynnie przejść do krótkiego omówienia:

- zdarzeń domenowych i integracyjnych,
- Outboxa,
- Inboxa i idempotencji,
- korelacji wiadomości,
- retry i kompensacji,
- kryteriów przejścia od modularnego monolitu do mikroserwisów.